

OOP.20252 - Homework Week 14

Ph.D. NGUYEN Thi Thu Trang, SoICT, HUST
TA: Dang Van Nhan, Nguyen Huu Hoang Hai Anh

General Submission Rules

- All assignments must be implemented individually.
- Students must submit all required files before **23:59 Sunday, May 31st, 2026**.
- Source code must compile and run successfully.
- **Students are required to follow Git commit conventions and release workflow guidelines as mentioned before on class.**
- Recommended commit convention reference:
<https://www.conventionalcommits.org/en/v1.0.0/>

Submission Requirements (applied to ALL exercises)

For each exercise, students must submit the following files:

1. Java source code project
2. Class Diagram:
 - Exported `.png` image
 - Original `.asta` file
3. Screenshots proving the application is running correctly
4. Reflection document `.pdf` file (2–300 words), include, **but not limited to**:
 - Main functionalities and brief explanation of the implementation
 - Design decisions and the rationale behind your design (How MVC was applied in the application?)
 - Problems, limitations, or design issues found (if any) in the previous version, and how those issues were addressed or improved in this version
 - How JavaFX properties, binding, and observable collections were used?

Important: Missing any of the four required submission components above will result in **0 points** for that exercise.

Repository and Submission Structure

Students will submit their work to the previously created repository, which is `00P.20252-[StudentID]`, where `StudentID` is the official student ID. Students are required to create a branch named `release/week14`. This branch must contain the following folder structure, where `hw14` contains submissions for the Date Display GUI 2 exercise.

Expected Repository Structure

```
00P.20252-[StudentID]
|-- (another folders of previous weeks)
|-- hw14/
    |-- src/
    |-- hw14.png
    |-- hw14.asta
    |-- screenshots/
    |-- hw14_reflection.pdf
```

1 Date Display GUI 2

1.1 Introduction

In the previous homework, students implemented a Date Display GUI application using Java AWT/Swing. In this homework, students will redesign the application using **JavaFX**, while applying the **MVC (Model–View–Controller)** architectural pattern and **JavaFX properties and binding**.

The application should retain its core functionality: displaying a date and allowing users to move it forward or backward by a specified number of days, months, or years. This version must be more reactive, modular, and maintainable.

1.2 Requirements

- The application must be implemented using **JavaFX** and must properly follow the **MVC architectural pattern**.
- The project should be organized into appropriate packages, such as `model`, `view`, and `controller`.
- The GUI must be built using JavaFX components. Students are encouraged to use **FXML** and Scene Builder.
- When the application starts, it must automatically display the current date.
- The application must allow users to choose the displayed date format **dynamically**. The GUI must provide a control, such as a `ComboBox`, for selecting different date formats, including but not limited to:
 - Vietnamese format: `dd/MM/yyyy`, for example `18/05/2026`.
 - US format: `MM/dd/yyyy`, for example `05/18/2026`.
 - European format: `dd.MM/yyyy`, for example `18.05.2026`.
 - Other reasonable date formats.

The displayed date must update automatically whenever either the selected date or the selected date format changes. This behavior must be implemented using JavaFX property binding, for example by binding the displayed date label's `textProperty()` to something that depends on both the selected date property and the selected date format property.

- GUI must provide appropriate controls, including:
 - a `DatePicker` for selecting the current displayed date,
 - a text field for entering a number `x`,
 - controls for choosing the operation direction: before or after,
 - controls for choosing the time unit: days, months, or years,
 - a button for applying the selected operation,
 - a label or message area for displaying validation errors,

- **a result area** showing the calculated date after the user applies the selected operation.
- The application must distinguish between the selected date and the result date. The selected date is the base date chosen by the user, while the result date is the calculated date shown after the Apply button is clicked. It must be based on the currently selected date, input number, operation direction, and time unit.
- Error messages must be displayed clearly when invalid input occurs, and date-related edge cases must be handled correctly. The Apply button must be automatically disabled when the input is invalid.
- Students are encouraged to use Java's `java.time.LocalDate` class for date calculations.
- Source code must be readable, modular, maintainable, and must compile and run successfully.

Grading Rubric

Academic Integrity Warning: TAs will review and grade all submissions carefully. Students must NOT copy, share, or reuse another student's work. If plagiarism or academic dishonesty is detected, both involved students will receive 0 points for both exercises.

Criteria	Weight
Application Functionality	30%
Software Design and Construction	60%
Submission Compliance	10%

Submission Compliance

- Correct file structure.
- Complete submission files.
- Proper naming conventions, clean code.
- Following deadline requirements.
- Proper Git usage and release process:
 - Use meaningful commit messages, follow Conventional Commits.
 - Obey to the release flow guideline.
 - Avoid committing unnecessary files.
- Late submission policy:
 - Submissions with commits made within **1 day after the deadline** will receive a 1-point deduction for all exercises.
 - Submissions with commits made **2 or more days after the deadline** will receive 0 points for all exercises.